FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Rey Joshua M. Delos Reyes
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	October 11, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Laboratory Task

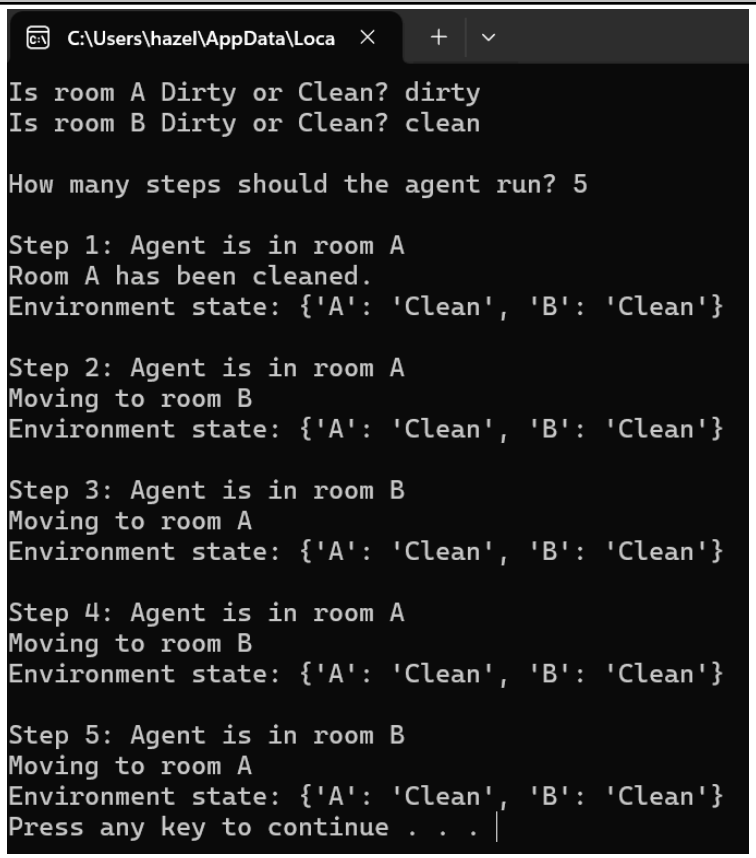
Pre-Lab Preparation

1. Review the Python concepts:
    - Classes and objects
    - If-Else statements
    - Loops (for loop)
    - Functions / methods
2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

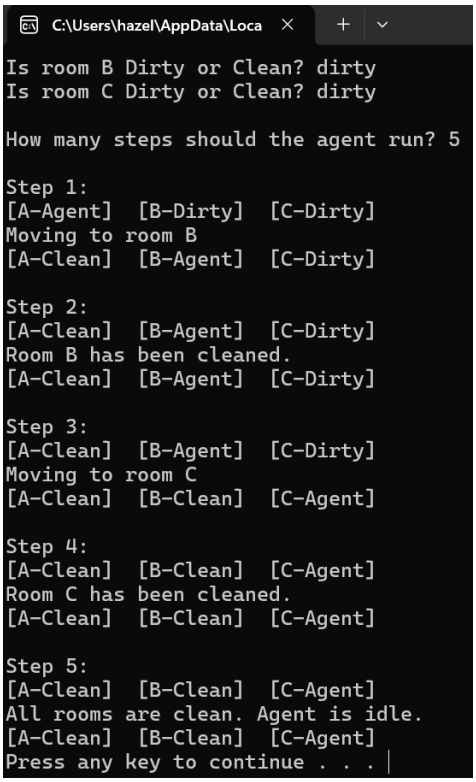
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import agentpy as ap


# Ask the user for the starting condition of a room.
# The function keeps asking until the user enters "dirty" or "clean".
def get_initial_state(room_name):
    while True:
                state = input(f"Is room {room_name} Dirty or Clean?
").strip().lower()

        if state == "dirty":
            return "Dirty"
        elif state == "clean":
            return "Clean"
        else:
            print("Please enter only Dirty or Clean.")


# Check if the selected room is dirty.
def is_room_dirty(rooms, room):
    return rooms[room] == "Dirty"


# Change the selected room's state to clean.
def clean_room(rooms, room):
    rooms[room] = "Clean"


# This class represents the vacuum cleaner agent.
class VacuumAgent(ap.Agent):

    # Set the agent's starting location to room A.
    def setup(self, **kwargs):
        self.location = "A"

    # Decide what the agent does during each step.
    def act(self, step_number):
```

```
        rooms = self.model.rooms

        print(f"Step {step_number}: Agent is in room {self.location}")

        # If the current room is dirty, clean it.
        if is_room_dirty(rooms, self.location):
            clean_room(rooms, self.location)
            print(f"Room {self.location} has been cleaned.")

        # If the current room is already clean, move to the other room.
        else:
            if self.location == "A":
                self.location = "B"
                print("Moving to room B")
            else:
                if self.location == "B":
                    self.location = "C"
                    print("Moving to room C")
                else:
                    self.location = "A"
                    print("Moving to room A")

        # Display the current condition of both rooms.
        print(f"Environment state: {rooms}\n")


# This class represents the whole simulation model.
class VacuumModel(ap.Model):

    # Create the room dictionary and create one vacuum agent.
    def setup(self, **kwargs):
        self.rooms = {
            "A": self.p.room_a,
            "B": self.p.room_b,
            "C": self.p.room_c
        }
```

```
        self.agent = ap.AgentList(self, 1, VacuumAgent)
        self.step_count = 0

    # Run one action for every model step.
    def step(self):
        self.step_count += 1
        self.agent[0].act(self.step_count)


# Get the initial state of each room from the user.
room_a = get_initial_state("A")
room_b = get_initial_state("B")
room_c = get_initial_state("C")

# Ask how many steps the simulation should run.
steps = int(input("\nHow many steps should the agent run? "))

# Store user input as model parameters.
parameters = {
    "room_a": room_a,
    "room_b": room_b,
    "room_c": room_c
}

# Create and run the model.
model = VacuumModel(parameters)
model.run(steps=steps, display=False)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The rules that the agent follows are that if the current room is dirty, it will clean it and change the status to clean. Once cleaned, it will go to the next room

2. The agent decides to clean the room once it is labeled as dirty, and it is going to move to another room once the status is clean
3. If both rooms are already clean at the start, it will go to the next room after those two rooms and clean that room
4. I would modify the code from hardcoding it as “A”, “B”, or “C” to a list so that the agent can navigate through the rooms through indexes